CHAPTER 4

Public Key Cryptography

Foundations of Cryptographic Security in Blockchain

Teguh Indra Bayu, Ph.D.

Faculty of Information Technology (FTI) — UKSW

TC789 — Teknologi Blockchain

Source: Mastering Blockchain, 2nd Ed. — Imran Bashir (Packt, 2018)

Learning Objectives

Chapter 4 — Public Key Cryptography

1. Explain the concept of asymmetric cryptography and how it differs from symmetric cryptography
2. Describe the mathematical hard problems underpinning RSA and ECC
3. Understand how RSA and ECC generate and use public/private key pairs
4. Explain the properties and applications of cryptographic hash functions
5. Describe how digital signatures (especially ECDSA) authenticate blockchain transactions
6. Recognize advanced constructs: Zero-Knowledge Proofs, homomorphic encryption, blind signatures

Chapter Overview

What this chapter covers

Topics Covered	Why It Matters for Blockchain
• Asymmetric cryptography fundamentals • RSA — key generation, encryption, decryption • Elliptic Curve Cryptography (ECC) • Cryptographic Hash Functions (SHA-256, SHA-3) • Message Authentication Codes (MACs) • Merkle Trees & Distributed Hash Tables • Digital Signatures & ECDSA • Advanced constructs: ZKPs, Homomorphic Enc.	• Blockchain transactions rely on public key cryptography for ownership and authorization • Hash functions create the tamper-evident "fingerprints" linking blocks in the chain • Digital signatures prove who authorized a transaction without revealing private keys • ECC (secp256k1) is the specific standard used in Bitcoin and Ethereum • Merkle trees enable efficient single-transaction verification (SPV)

Symmetric vs. Asymmetric Cryptography

The key distribution problem — and its solution

Symmetric Cryptography

- Same key for encryption & decryption
- Fast — suitable for bulk data encryption
- Problem: key distribution requires a secure channel
- Examples: AES, DES, 3DES

Asymmetric Cryptography

- Two keys: Public key (shared openly) + Private key (kept secret)
- Slower — used for key exchange, not bulk data
- Solves the key distribution problem
- Examples: RSA, ECC, ElGamal, DSA

Mathematical Foundations

Three hard problems that make public key cryptography secure

1. Integer Factorization

- Easy: multiply two large prime numbers
- Hard: factor the product back into primes
- n = p × q — computationally infeasible to reverse
- Foundation of: RSA

2. Discrete Logarithm

- Easy: compute g^x mod p = y
- Hard: find x given only y
- One-way function — cannot run in reverse
- Foundation of: Diffie-Hellman, DSA

3. Elliptic Curve (ECDLP)

- Discrete log problem on curve points
- Curve: $y^2 = x^3 + ax + b$ over finite field
- Hardest known per bit of key size
- Foundation of: ECDSA, ECDH (Bitcoin, Ethereum)

RSA Cryptosystem

Rivest–Shamir–Adleman, 1977 — integer factorization

Key Generation Steps

1. Select two large prime numbers p and q
2. Compute modulus n = p × q
3. Choose co-prime e (1 < e < (p-1)(q-1))
4. Public key = (n, e)
5. Private key d = inverse of e mod (p-1)(q-1)

Encryption & Decryption

Encryption

$$C = P^e \bmod n$$

P = plaintext, e = public exponent, n = modulus

Decryption

$$P = C^d \bmod n$$

d = private key, C = ciphertext

Elliptic Curve Cryptography (ECC)

Smaller keys, same security — the choice of blockchain

Why ECC for Blockchain?

- Based on ECDLP — hardest known problem per key bit
- Much smaller keys than RSA for equivalent security
- Faster key generation — critical for blockchain at scale
- Used in Bitcoin (secp256k1) and Ethereum
- Curve equation: $y^2 = x^3 + ax + b$ over finite field Fp
- secp256k1: specific curve chosen for Bitcoin

RSA vs. ECC Key Size Comparison

RSA (bits)	ECC (bits)	Security
1024	160	80-bit
2048	224	112-bit
3072	**256**	**128-bit ★**
7680	384	192-bit
15360	521	256-bit

★ Bitcoin uses secp256k1 — a 256-bit ECC curve

Cryptographic Hash Functions

One-way fingerprints for data integrity

Definition & Security Properties	
Takes any-length input → fixed-length digest (128–512 bit). Keyless, one-way, deterministic.	
Compression:	Any input → fixed-length output
Fast to compute:	Efficient one-way function
Preimage resistance:	Cannot reverse h(x) to find x
2nd preimage resist.:	No x' ≠ x with h(x') = h(x)
Collision resistance:	No two inputs → same hash

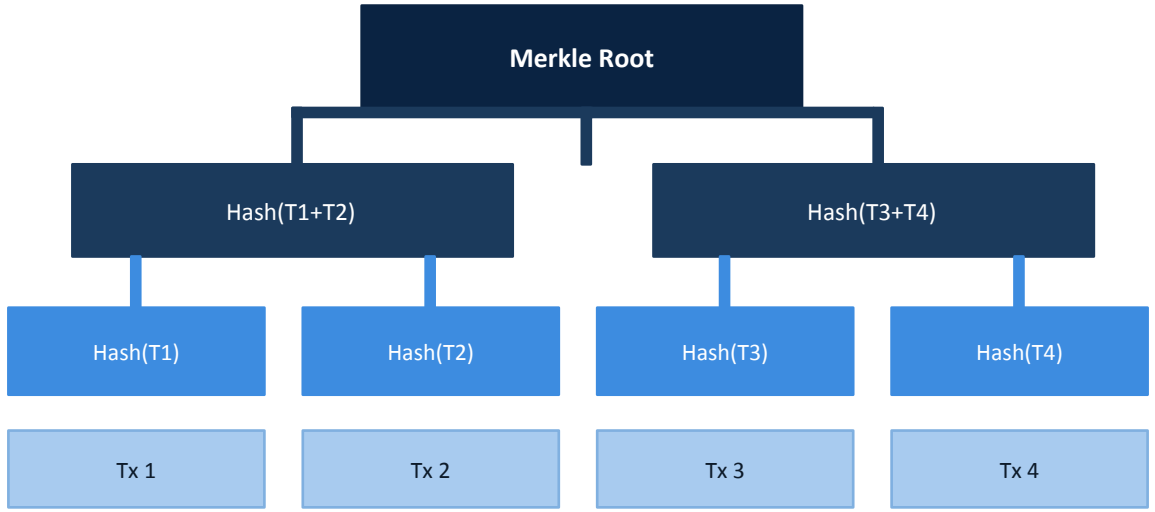
Common Algorithms & Blockchain Use		
MD5	128-bit	Deprecated — insecure
SHA-1	160-bit	Deprecated — insecure
SHA-256	256-bit	Bitcoin PoW & Merkle roots
SHA-3 / Keccak	256-bit	Ethereum (Keccak variant)
RIPEMD-160	160-bit	Bitcoin address generation

Avalanche effect: change 1 input bit → completely different hash output

Merkle Trees

Efficient and tamper-evident transaction verification in blockchain

Merkle Tree Structure



Key Properties & Blockchain Uses

- Binary tree — leaves hold data hashes; internal nodes hold hash of children
- Merkle root = single hash summarizing ALL transactions in a block
- Changing any transaction changes the entire path up to the root
- SPV: verify one transaction without downloading the full blockchain
- Patricia trees (Ethereum) extend this for key-value state storage
- DHTs apply hashing to distribute data across P2P networks

Digital Signatures

How blockchain proves ownership and authorizes transactions

Signing & Verification Process

1. **Hash the transaction data**
   SHA-256 produces a fixed-length digest
2. **Sign the hash**
   Using sender's private key (ECDSA)
3. **Broadcast tx + signature**
   Signature (r, s) appended to transaction

Verification: Network nodes use sender's public key to confirm the transaction is authentic and unmodified.

ECDSA & Signature Properties

Key Pair:	Private key d (random int); Public key B = d × A (point on curve)
Sign:	Choose ephemeral key Ke; compute R = Ke × A; signature = (r, s)
Verify:	Check Xp = r mod q using public key and signature parameters
Authenticity:	Only the private key holder can produce a valid signature
Unforgeability:	No other party can replicate a legitimate signature
Non-repudiation:	Signer cannot deny having authorized the transaction

Message Authentication Codes (MACs)

Keyed hash functions for data integrity and authentication

What are MACs?

- Keyed hash functions using a shared symmetric key
- Provide: data integrity + data origin authentication
- Cannot provide non-repudiation (symmetric — either party could create the MAC)
- CBC-MAC: uses AES (or other block cipher) in CBC mode
- HMAC: hash-based MAC with inner and outer padding

```
HMAC = h(k || x)    or    h(x || k)
```

Hash Functions in Blockchain

PoW	Proof of Work: miners compute SHA-256(SHA-256(block)) to find valid hash
Addr	Bitcoin addresses: RIPEMD-160(SHA-256(public key))
Merkle	Merkle trees: SHA-256 hashes link all transactions in a block
Block	Block linking: each header contains the previous block's hash
ETH	Ethereum: uses Keccak-256 (SHA-3 variant) throughout its protocol

Advanced Cryptographic Constructs

Specialized tools relevant to blockchain privacy and scalability

Homomorphic Encryption

- Compute on encrypted data without decryption
- Partial HE: supports addition OR multiplication
- Full HE (Craig Gentry, 2009): supports both
- Application: privacy-preserving blockchain computation

Zero-Knowledge Proofs (ZKPs)

- Prove a statement is true without revealing it
- Properties: Completeness, Soundness, Zero-knowledge
- ZK-SNARK used in Zcash cryptocurrency
- Enables private transactions on a public blockchain

Blind Signatures

- Sign a message without revealing its content
- Introduced by David Chaum (1982)
- Foundation of digital cash schemes
- Based on RSA with message blinding

Signcryption & Encoding Schemes

- Signcryption: signature + encryption in one step
- Cost(signcryption) < Cost(sign) + Cost(encrypt)
- Base64: binary-to-text encoding
- Base58: Bitcoin encoding — no ambiguous chars (0, O, l, I)

Key Terms & Definitions

Quick reference for Chapter 4 terminology

Public Key	Freely shared key; used to encrypt messages or verify digital signatures
Private Key	Secret key; used to decrypt messages or create digital signatures
RSA	Asymmetric algorithm based on integer factorization of large prime numbers
ECC	Elliptic Curve Cryptography — based on ECDLP; requires smaller keys than RSA
secp256k1	Specific 256-bit ECC curve used in Bitcoin and many other blockchains
Hash Function	One-way function producing a fixed-length digest from any-length input
Merkle Tree	Binary hash tree enabling efficient, tamper-evident data verification
ECDSA	Elliptic Curve Digital Signature Algorithm — standard for blockchain tx signing
ZKP	Zero-Knowledge Proof — prove a statement true without revealing the statement itself
HMAC	Hash-based MAC — provides message integrity using a shared secret key

Benefits vs. Limitations

Public key cryptography in blockchain context

Benefits

- No shared secret key to transmit — solves key distribution
- Non-repudiation: signer cannot deny authorizing a transaction
- ECC provides strong security with very small keys (256-bit)
- Enables trustless ownership — anyone can verify signatures
- Hash functions make data tampering computationally infeasible

Limitations & Challenges

- Slower than symmetric cryptography — not for bulk data encryption
- Private key loss = permanent asset loss (no recovery mechanism)
- Key management complexity — secure storage is critical
- Quantum threat: Shor's algorithm can break RSA and ECC
- Post-quantum cryptography required for long-term security

Key Takeaways

Chapter 4 — Public Key Cryptography

1. Asymmetric cryptography uses a key pair (public + private) — no shared secret needs to be transmitted.
2. RSA relies on integer factorization; ECC relies on the elliptic curve discrete log problem.
3. ECC requires only 256-bit keys (vs. 3072-bit RSA) for equivalent security — critical for blockchain efficiency.
4. Hash functions are one-way, collision-resistant fingerprints — SHA-256 powers Bitcoin's Proof of Work.
5. Merkle trees enable tamper-evident transaction linking and efficient single-transaction verification.
6. ECDSA digital signatures prove transaction ownership without revealing the private key.
7. secp256k1 is the ECC curve used in Bitcoin; ZKPs and homomorphic encryption address blockchain privacy.

Discussion & Q&A

Connect the theory to real blockchain systems

Q If you lose your private key, what happens to your cryptocurrency? Is there a recovery mechanism?

Q Why does Bitcoin use ECC (secp256k1) instead of RSA? What practical advantages does this bring?

Q What would happen if hash functions could be reversed? How would this break blockchain security?

Q How does a digital signature prevent someone from spending another person's Bitcoin?

Q Zero-Knowledge Proofs allow private transactions on a public chain — can you think of a use case?